

Abstract

This research work is all about creating a strong sentiment algorithm that focuses on Amazon book reviews. We use some cool techniques here. The process includes cleaning, designing models, & testing different approaches. We explore deep learning models like bidirectional LSTM networks, adjusting hyperparameters to improve how well it performs. The work also looks at how the length of reviews, the way they are written, & specific factors in the domain might change accuracy. The results give us a look into what customers like and their feedback, pushing forward sentiment analysis while giving useful insights for companies and researchers who are studying how people feel about online reviews.

Table of Contents

1. Introduction to Text Classification.....	1
1.2. Sequential Model Architecture.....	1
2. Methodology.....	1
2.1. Data Pre-Processing.....	2
2.2. Model Building.....	2
2.3. Training of the Model.....	2
2.4. Evaluation of the Model.....	4
3. Final Discussion.....	4
References.....	5

1. Introduction to Text Classification

I used the Amazon Book Review dataset for classification of text. This dataset contains user reviews, ratings, & summaries. The main aim was to sort out the sentiments in those book reviews as either positive or negative. (Anon., n.d.)

1.2. Sequential Model Architecture

In this sequential model meant for text classification, we first break down the text data — it's tokenized and padded before turning into vectors using something called an embedding layer. A bidirectional LSTM lets us gather contextual info while a global max pooling layer simplifies things. Dense layers come with ReLU activation & L2 regularization to find patterns in the data; dropout layers help us avoid overfitting. Lastly, the output layer gives us sentiment probabilities using sigmoid activation. The model is trained with binary cross-entropy loss & the Adam optimizer while keeping an eye on progress through callbacks.

2. Methodology

We took the Amazon Book Review dataset and prepped it by removing punctuation, changing all words to lowercase, getting rid of stop words, and lemmatizing using NLTK. This ensured the text was ready for modeling purposes. A sequential neural network was created using TensorFlow and Keras that had an embedding layer along with a bidirectional LSTM to get context & a global max pooling layer to lower dimensionality. Dense layers featuring ReLU activation, dropout techniques, and L2 regularization worked together to uncover patterns and stop overfitting. Sentiment predictions were done using sigmoid activation as we trained the model with Adam optimizer & binary cross-entropy loss while fine-tuning hyperparameters for top performance.

The steps involved cleaning up the data first, building up the model architecture next and then training it while adjusting hyperparameters later on—to build an effective model for analyzing sentiments in Amazon book reviews.

2.1. Data Pre-Processing

In this first phase of preparing data, I used several methods to make text better organized for analysis. Punctuation got wiped out; all text turned lowercase too! I used lemmatization with NLTK to make words simpler & consistent. To see which words popped up more often, I created a word cloud. TensorFlow Keras' Tokenizer class was handy here; it changed text into numerical sequences and padding was added to keep sequence length consistent throughout. By doing all these things, we made sure that our text data was clean & ready for deeper analysis and modeling!

2.2. Model Building

In the text classification model, several layers together to handle text data. First, there's an Embedding Layer. This turns text into dense vectors. Next, we have the Bidirectional LSTM Layer, which captures context from both directions. Then comes the GlobalMaxPooling1D Layer that helps reduce dimensionality. After that, are Dense Layers with LU activation to find complex patterns. Finally, a Dropout Layer kicks in to prevent overfitting. The Output Layer uses sigmoid activation for binary classification tasks. Important hyperparameters used include embedding dimension, LSTM units, dropout rate, regularization strength, batch size, number of epochs, the Adam optimizer, and binary_crossentropy loss.

2.3. Training of the Model

To train this model, we used binary cross-entropy loss function—it's great for binary classification tasks like sentiment analysis! The Adam optimizer was chosen because it is known for its efficiency & adjustable learning rate; it was set to run over four epochs. At first, both the training and validation losses went down, but after a few epochs, the training loss kept dropping while validation loss crept up a bit. This means there could be some overfitting happening. This gap suggests that maybe the model is just getting too familiar with the training data. To fight against this issue, we use regularization techniques like L2 regularization & dropout to help reduce overfitting while boosting how well the model generalizes.

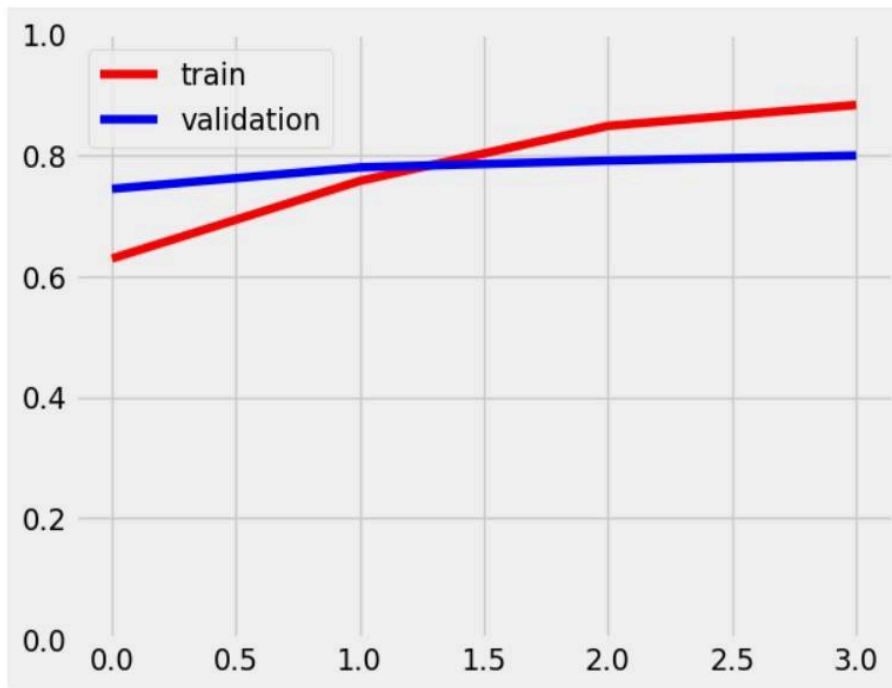


Figure 1 Plot between train accuracy and validation accuracy

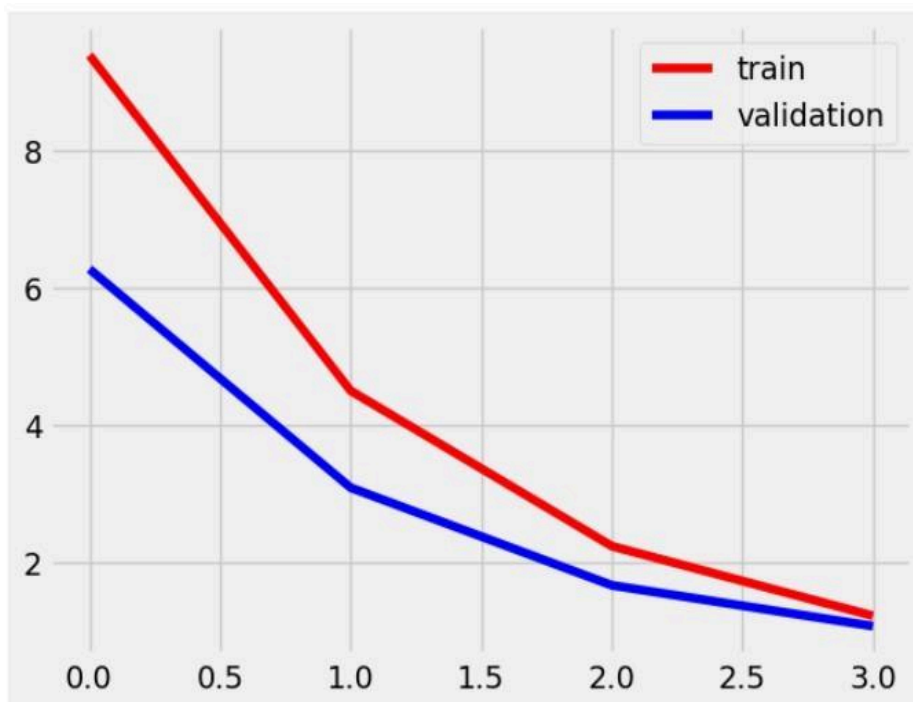


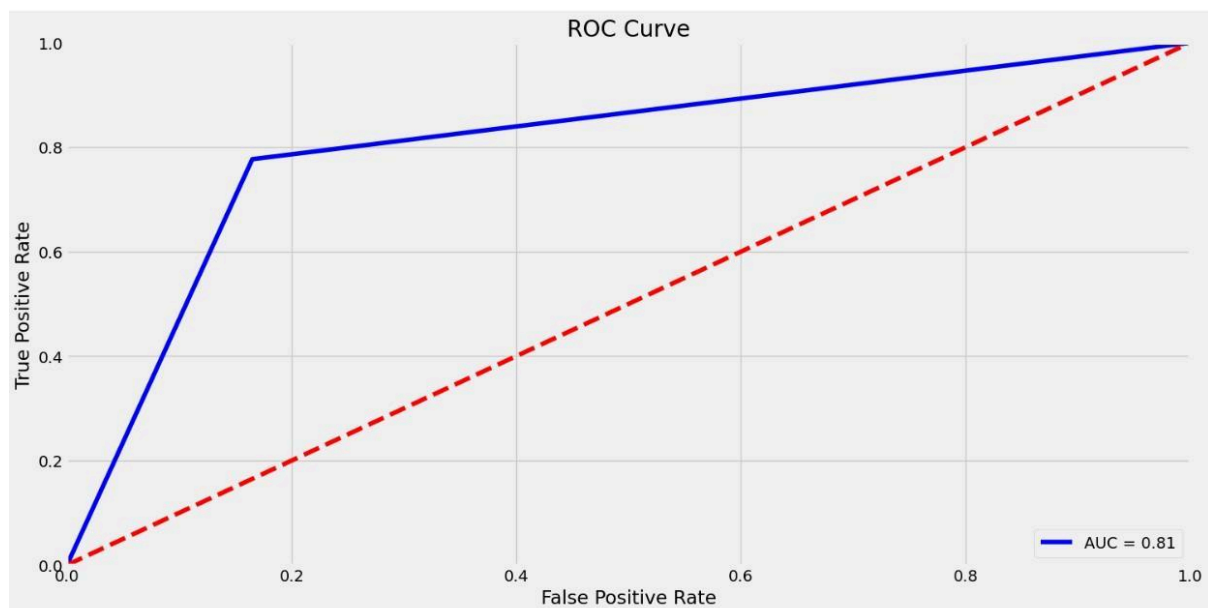
Figure 2 Plot between train loss and validation loss

2.4. Evaluation of the Model

When tested with simulated data for sentiment analysis, the model did quite well! It hit about 80.56% accuracy overall. It shows balanced performance too; recall & F1-score values sit between 0.78 and 0.83 for both negative & positive classes. This suggests it's pretty good at finding positive sentiments along with a decent balance between true positives & false positives—that's important stuff! The confusion matrix tells us a bit more about how well the model can tell apart positive and negative sentiments too—there are a few more false positives than false negatives slightly showing where it can improve! Overall, this model seems strong for practical use in classifying sentiments from unseen test data.

3. Final Discussion

This sentiment analysis model does a great job of sorting out positive and negative reactions in Amazon book reviews; it gives good insights into what customers think! But there's always room to make things better—like cutting down on those false positives and boosting overall accuracy even more! Future efforts could focus on fine-tuning this model or maybe looking into new methods for preparing the data while also testing some different feature engineering techniques out there! All in all, this model looks promising for real-world tasks like figuring out sentiment in e-commerce settings—it's a big help for making decisions!



References

Anon., n.d. *Amazon Books Reviews*. [Online]

Available at:

<https://www.kaggle.com/datasets/mohamedbakheta/amazon-books-reviews>

[Accessed 2024].